



AKADEMIA GÓRNICZO-HUTNICZA IM. STANISŁAWA STASZICA W KRAKOWIE

Wydział Informatyki

Projekt dyplomowy

*Biblioteka dla analiz stabilności wyników opartych o
uczestnictwo w wyborach komitetów i budżetach obywatelskich*

*Library for Robustness Analysis Based on Participation in
Multiwinner Voting and Participatory Budgeting*

Autorzy:	Kyrylo Iakymenko
Kierunek studiów:	Informatyka
Opiekun pracy:	prof. dr hab. inż. Piotr Faliszewski

Kraków, 2026

Contents

1	Project Goals and Vision	5
1.1	General Introduction	5
1.2	Mathematical Model	6
1.2.1	Greedy Approval Voting (Greedy-AV)	6
1.2.2	Plurality	7
1.2.3	Phragmén's Methods	8
1.2.4	Method of Equal Shares (MES)	8
1.3	Real World Applications	10
1.4	Overview of Existing Solutions	11
1.5	Final Product Vision	12
2	Functional Scope	13
2.1	Closer Look at Provided Functionality	13
2.2	Project Structure	13
2.3	Functional Requirements	14
2.3.1	Must	14
2.3.2	Should	15
2.3.3	Could	15
2.3.4	Won't	16
2.3.5	Notes on Prioritization and Acceptance Criteria	16
2.4	Quality Assurance	16
2.4.1	QA Objectives	16
2.4.2	Testing Strategy	17
2.4.3	Test Data and Reproducibility	18
2.4.4	Code Quality and Static Analysis	18
2.4.5	Documentation and Examples Testing	18
2.4.6	Release and Packaging Tests	18
2.4.7	Acceptance Criteria and Metrics	18
3	Selected Implementation Aspects	19
3.1	High-Level Architecture and System Overview	19
3.1.1	Main Modules	19
3.1.2	Conceptual Structure of the Library	19
3.2	Technology Stack	20
3.3	Component Overview and Interfaces	20
3.3.1	Core Module	20
3.3.2	Robustness Module	21
3.3.3	Visualization Module	21

3.4	Development Environment	21
3.5	Quality Assurance	21
3.6	Technical Challenges	22
3.7	Summary	22
4	Work Organization	23
4.1	Project Characterization and Development Model	23
4.2	People Involved	23
4.3	Organization of Work and Tools	24
4.4	Engineering Practices	24
4.5	Timeline and Milestones	24
4.6	Deliverables and Acceptance Criteria	24
4.7	Use of AI tools	25
4.8	Concluding Remarks on Organization	25
5	Project Results	26
5.1	Theoretical Results	26
5.2	Practical Experiments	33
5.2.1	Greedy-AV Outcome Visualization	33
5.2.2	Analysis of SRM for Real-Life Instances	35
5.2.3	Larger Instances	37
5.2.4	Comparison of SRM Measure of Elections in Poland and Worldwide.	39

Chapter 1

Project Goals and Vision

The goal of this project is to create a Python library for analyzing the stability of outcomes in committee elections (multiwinner voting) and participatory budgeting. In typical PB elections, voter turnout is usually between 5% and 10%. This means that the results could change significantly if many more people cast their votes. However, we hope that those who participated in the vote are, to some extent, representative of the entire population. Therefore, it would be useful to calculate the probability of individual candidates/projects winning, assuming that a certain number of votes are cast and these votes are sampled according to the probability distribution derived from the actually cast votes.

We will develop a library that will provide tools for implementing the robustness analysis described above. The library will include functions for simulating elections (including implementations of relevant voting rules), code for visualizing results, and usage examples. Additionally, we will enhance the project with theoretical proofs concerning the complexity and/or other properties of the algorithms used.

1.1. General Introduction

Collective decision making is a cornerstone of modern social and political life. Beyond national elections, multiwinner voting and participatory budgeting (PB) are used in a wide variety of real-world contexts: from selecting representative committees and allocating public funds, to choosing collections of public goods, or deciding which proposals receive limited grant money. In all these settings, the outcome is a set rather than a single winner, and the quality of the outcome is judged by how well it reflects the preferences and needs of the affected population.

A key practical challenge in these domains is low and potentially biased participation. In many PB processes and local consultations, turnout is often very small (empirical studies commonly report participation in the single-digit or low double-digit percentages; in some contexts, turnout may be as low as 5-10% range). When only a small fraction of the population votes, the observed profile of ballots is a noisy and possibly unrepresentative sample of the underlying population preferences. This raises questions that motivate the work in this thesis:

1. To what extent can the winners (selected committee members or funded projects) be trusted as robust or stable under plausible changes in participation?
2. Can we efficiently calculate a type of uncertainty measure that would accurately answer the above question?

3. What practical algorithms can be used in real world instances to assess and optimize project performance in such election instances?

This thesis develops computational tools, as well as a form of theoretical background, that allow for the study of such questions.

1.2. Mathematical Model

Let us now present the basic mathematical model of approval-based committee elections. Our primary source for this section is a survey article by Lackner and Skowron [2023](#).

First, fix non-empty finite sets of candidates C and voters V , and let $k \in \mathbb{N}$, with $k \leq |C|$. We will interpret k as the size of the committee to be selected. We can then define an approval profile as a function $A : V \rightarrow P(C)$, which assigns to each voter the set of candidates they approve of. The field of approval-based committee elections involves studying such functions and their properties. We can now define an instance of election $E = (A, k)$, where A is a given approval profile. A voting rule should (informally) correspond to some way of choosing a committee of size k from a given election instance. Note that the selection of a committee need not be unique and that ties may occur in elections that cannot be resolved while treating all candidates and voters equally (a property that is often desirable in practice). It could be in principle formally defined as a function-like formula on the (proper) class of all election instances that return the set of all possible outcomes, but we will not concern ourselves with such formalisms in what follows.

We now move on to participatory budgeting. Participatory budgets naturally generalize approval based committee elections by allowing for variable “prices” of a candidate. The rules of the approval-based committee select exactly k candidates, while in PB we set a budget and select candidates based on it (typically, there is no limit on the number of candidates to be selected), each of which has its own fixed cost. Formally, each election instance should now come with a cost function $c : C \rightarrow \mathbb{R}_{\geq 0}$ and a budget $B \in \mathbb{R}_{\geq 0}$. As in approval based committee elections, we consider different rules to select a subset of selected candidates $\pi \subseteq P$, such that $\sum_{i \in \pi} c(i) \leq B$. In fact, one can see an approval election as a PB instance with budget B set to committee size k and each project resembling candidate with “unit” cost (i.e., $c = 1$). This formalism directly models real-world budget allocation situations, and thus has many practical applications. For a more comprehensive survey of PB, see the work of Rey et al. [2025](#).

We now define the voting rules that will be studied in this work and discuss some of the main properties of these rules that we will study in this work.

1.2.1. Greedy Approval Voting (Greedy-AV)

Greedy Approval Voting (Greedy-AV) is a simple, widely used sequential rule for selecting a committee based on approval ballots. Intuitively, the rule repeatedly picks the candidate (or project) that currently has the largest number of approving voters until the committee reaches the desired size (or no further projects can be added under the budget).

Formally, let $E = (A, k)$ be an approval election with the voter set V , the candidate set C , the approval profile $A : V \rightarrow P(C)$, and the size of the committee k . For $W \subseteq C$ denote by $W(c)$ the number of voters who approve candidate c . The Greedy-AV rule can be described as follows

Budgeted (PB) variant. In participatory budgeting with costs $c(\cdot)$ and budget B , a natural Greedy-AV variant repeatedly selects the most approved project that fits the remaining budget.

Many natural variants exist, but we will not discuss them here (see the works of Lackner and Skowron 2023 and Rey et al. 2025).

Remarks and Properties.

- The rule tends to favor candidates (or projects) that are approved by the largest majority of voters, often at the expense of proportional representation for minorities.
- Greedy-AV treats every additional selected candidate approved by the same voter with full weight, which can lead to over-representation of cohesive majority.
- Outcomes can be sensitive to tie-breaking; different tie-breaking rules (deterministic, lexicographic, random) may produce substantially different committees in practice.
- The general simplicity allows for a comprehensive study of different types of measures in real world instances, which may be significantly harder on more complicated types of voting rules (see the work of Boehmer et al. 2025).

1.2.2. Plurality

Plurality, while usually not considered separately and is seen as the simplest variant of Greedy-AV, nevertheless we present it here since it plays a crucial role in our study of the SRM measure, which will be presented in later chapters (see the Chapter 5). Plurality can be described as follows: each voter may cast a single vote (or, equivalently, in our approval framework, may approve at most one candidate/project), and the rule selects the k candidates with the largest number of votes (or — in a sequential/budgeted setting — repeatedly selects the single most-voted candidate that fits the constraints). Again we note that it is a special case of the above-mentioned Greedy-AV, with restrictions on the number of candidates selected by a single voter.

Committee/selection description. Given an election $E = (A, k)$ where each voter v satisfies $|A(v)| \leq 1$, define for each candidate c the plurality score

$$\text{pl}(c) = |\{v \in V : A(v) = \{c\}\}|.$$

Plurality selects the k candidates with the highest $\text{pl}(c)$ (ties broken according to a chosen rule). In the PB setting, a natural greedy plurality approach repeatedly funds the project with the highest plurality score that fits the remaining budget.

Remarks and Properties.

- Plurality is well-suited for baseline tests on real world instances.
- In general may share the same problems as Greedy-AV
- Its overall simplicity allows for the efficient implementation of the most difficult measures in practice. It may be used to assess project performance, when we are unable to do so with respect to a chosen rule, due to its computational complexity.

Properties of the so called Sampling Robustness Measure (to be defined in Chapter 5) will be studied in the cases of both of the above mentioned rules (as well as their variants) theoretically and in practice using implemented algorithms, whenever it is possible to do so efficiently.

We will also concern ourselves with two additional voting rules.

1.2.3. Phragmén’s Methods

Phragmén’s family of rules approaches proportional representation by *balancing the load* among voters. Intuitively, every selected member of the committee must be “paid for” by the voters who approve it, and Phragmén chooses a committee that spreads this payment as evenly as possible across the electorate.

Sequential Phragmén Sequential Phragmén (often used in practice) is a polynomial-time greedy procedure that can be viewed as a continuous-time process:

1. Initialize all voter loads $L_v \leftarrow 0$ and $W \leftarrow \emptyset$.
2. While $|W| < k$:
 - (a) For each remaining candidate $c \notin W$, consider increasing the loads of all voters in $A(c)$ at the same uniform rate until the total additional load contributed to c reaches 1; this would require time

$$t_c = \frac{1}{|A(c)|}$$
 - (b) Select the candidate c that can be “paid for” earliest (i.e., minimizes the time until a unit of load is gathered under the current schedule), add c to W , and update the voter loads accordingly.

Sequential Phragmén can be implemented efficiently (polynomial time) and is therefore attractive for large instances.

Remarks and Properties.

- Phragmén’s rules emphasize *fair sharing* of representation, which could be desirable in real world applications.
- *Adaptation to participatory budgeting.* For PB with costs $c(\cdot)$ and budget B a natural Phragmén-style extension is to require that each selected project p consumes $c(p)$ units of load, i.e. replace the per-candidate constraint $\sum_v x_{v,c} = 1$ by

$$\sum_{v \in V} x_{v,p} = c(p) \quad \text{for each selected project } p,$$

and require $\sum_{p \in W} c(p) \leq B$. Sequentially, one may grow voter loads until a project’s full cost is covered, and then select that project, taking costs into account in the stopping condition.

1.2.4. Method of Equal Shares (MES)

The *Method of Equal Shares* (MES) is a proportional rule originally proposed in the participatory budgeting literature that allocates the available budget by first giving every voter an equal share of the budget and then greedily “buying” projects using voters’ remaining shares. MES is particularly natural in PB settings with non-uniform project costs, but it also may be used in unit-cost elections.

Model. Let $E = (A, C, V, c, B)$ be a participatory budgeting instance where V is the set of voters, C the set of projects, $A : V \rightarrow P(C)$ the approval profile, $c : C \rightarrow \mathbb{R}_{>0}$ the cost function and $B \geq 0$ the total budget. Denote $n = |V|$. In MES, each voter $v \in V$ is initially endowed with a personal budget

$$b_v^{(0)} = \frac{B}{n}.$$

During the algorithm, voters spend parts of their personal budgets to fund projects they approve; a project is selected once the supporters can jointly pay its full cost under the rule's per-supporter contribution constraints.

Sequential MES (Algorithmic Description). Let b_v denote the remaining budget of voter v in a given step (initially $b_v = b_v^{(0)}$). For any project $p \in C$ and the remaining current budgets $(b_v)_{v \in V}$ define the *threshold* t_p as the smallest nonnegative real number that satisfies

$$\sum_{v \in A(p)} \min(b_v, t_p) \geq c(p),$$

where $A(p) = \{v \in V : p \in A(v)\}$ is the set of supporters of p . Intuitively, t_p is the per-supporter contribution level at which supporters would together be able to cover p 's cost, given their remaining budgets.

The sequential MES procedure repeatedly selects projects as follows:

1. Initialize $W \leftarrow \emptyset$ and $b_v \leftarrow B/n$ for every $v \in V$.
2. There exists a project $p \in C \setminus W$ with $t_p < \infty$ and $\sum_{x \in W} c(x) < B$:
 - (a) For every $p \in C \setminus W$ compute the threshold t_p (with respect to current budgets b_v).
 - (b) Choose a project $p^* \in \arg \min_{p \in C \setminus W} t_p$ (break ties by a predefined tie-breaking rule).
 - (c) Select p^* and fund it by deducting contributions from its supporters.
3. Return the selected bundle W (note: in many variants the algorithm stops when no affordable project remains, which may leave some budget unspent).

Interpretation. MES can be viewed as giving every voter an equal voice (an equal share of the budget) and then charging supporters equally for projects they collectively approve. The threshold t_p captures how “expensive per-supporter” a project is given current remaining budgets, and selecting the project with minimal t_p implements a form of cost-aware proportionality: projects that can be funded by small equal contributions from many supporters are preferred.

Remarks and Properties.

- The sequential MES runs in polynomial time, which makes it practical for real PB instances.
- A known characteristic of MES is that it may leave some portion of the budget unspent (it is not always exhaustive). There exist variants and post-processing heuristics that address this issue.

We will study the above mentioned rules by extending the measures of robustness of performance of a single project, which were defined in the work of Boehmer et al. 2025, to the cases where the project in question loses the original election.

We will mainly concern ourselves with PB election instances as they generalize approval-based committee elections as well as are better suited to most real world applications. In the PB setting, we may consider elections with a relatively low voter turnout, as was discussed in the introduction (Section 1.1). The central modeling assumption for a general study of such instances is that the ballots cast actually represent a general opinion accurately, i.e. if we treat them as probability distribution over voter types in the full population, we would get similar results even in the case if more people had cast their votes. Concretely, let C be the set of all candidates or projects and let $A = \{a_1, \dots, a_m\}$ be the multi-set of ballots observed in an election with m votes. From A one can form an empirical distribution p_A over the types of ballot. Suppose a larger turnout of n voters would occur and those n ballots would be drawn at random from a uniform distribution of votes p_A . Under this sampling model, we can ask probabilistic questions about any voting rule $\mathcal{R}$: for each candidate or project $c \in C$, compute

$$\Pr_{B \sim p_A^n} [c \in \mathcal{R}(B)],$$

the probability that c belongs to the winning committee (or is funded) when n ballots are drawn from p_A . This is one of the questions of this work and is essentially related to understanding what would be the most effective way to measure the project performance under a given voting rule. We may refer to questions of this type as studying “local stability” of the project in an election instance as it is ultimately a question of performance of a single project. We will mostly develop functions to assess such project performance based on heuristics, rather than computing the probability of this project being selected directly.

The second question we will touch upon is some sort of “global stability” of election instances. That is, given an election instance with the set of selected candidates W and treating the ballots observed so far as a uniform distribution, we will study the question of determining the following probability:

$$\Pr_{B \sim p_A^n} [W = \mathcal{R}(B)],$$

that is the probability that the same set of candidates will be selected in election instances that will use votes sampled from the original election. It seems quite natural to use this model since we may interpret such probability as a measure of robustness of an election instance. We will study this question in the cases of Greedy-AV and Plurality (see Section 1.2 for definitions and Chapter 5 for results) as well as their variants.

1.3. Real World Applications

There are several reasons why such an analysis is valuable. First, it allows for decision-making in a more informed manner, based on data supplied by various tests and measures of robustness. Second, probabilistic outcome estimates can detect fragility (situations in which small changes in turnout or small sampling fluctuations produce radically different outcomes), enabling one to take corrective action (e.g., run additional rounds of outreach or change the voting rule). In practice, this application seems to be extremely important, as it is crucial that the resources allocated maximize the overall satisfaction. Third, these tools help diagnose potential representativeness issues: if the probabilities are highly sensitive to modest changes in the sam-

pling distribution, the organizer may need to address sampling bias before treating the result as legitimate.

From a methodological perspective, two complementary approaches are useful. The empirical/computational approach relies on simulations and resampling techniques (using the empirical distribution p_A) to approximate the probabilities of the results and produce visual summaries. This approach is flexible and works with arbitrary voting rules, including both approval-based and ordinal multi-winner rules, as well as budget-limited PB rules. We also study the theoretical side of things, which allows for a more robust study of computational complexity of algorithms and provides the guaranty that computing studied measures is indeed hard. Both perspectives are important: simulations provide practical diagnostics on real instances, while theory explains when and why stability can or cannot be certified efficiently. We will focus on both approaches that help us compute certain measures and assess project performance in practice, as well as theoretical investigations that provide us with algorithms in some cases or proofs of hardness in others.

The objective of the software developed in this project is to make the analysis described above accessible and reproducible. The library will provide the following.

- Functions to estimate outcome probabilities and assess project performance in real-world PB instances.
- Utilities for stability analysis that will help to assess how low voter-turnout may affect the overall satisfaction with election results and measure, to which extent it is prone to manipulation.
- A suite of examples and benchmarks drawn from synthetic and real-world instances demonstrating the usage of implemented algorithms and measures in practice.
- Theoretical results concerning the complexity of determining the Sampling Robustness Measure in the cases of Greedy-AV, Plurality, and their variants.
- Implementation of Greedy-AV, that is compatible with existing solutions such as ‘pabutools’.

1.4. Overview of Existing Solutions

One of the best Python libraries in the field of computational social choice is [abcvoting](#) (Lackner, Regner, et al. 2023). This library provides implementations of a wide range of approval-based multiwinner voting rules, including Approval Voting, Proportional Approval Voting (PAV), sequential PAV (seq-PAV), and many others. It supports both unweighted and weighted elections and allows for experimenting with different voting rules in a unified framework. In addition, [abcvoting](#) includes features for computing winning committees, handling ties, and performing some basic result analysis.

Another relevant library is [pabutools](#), which focuses specifically on participatory budgeting (PB). It provides implementations of PB rules, including those based on the satisfaction of voters, such as the Max-Satisfaction and Greedy rules, as well as rules inspired by proportional representation. The library supports various budget constraints and cost functions and offers tools for analyzing how different rules select projects under limited resources.

To study real world election we will repeatedly use the **Pabulib** (see the work of Stolicki et al. 2020), a Participatory Budgeting library, that has many real world instances and allows for

seamless integration with Pabutools, which has been tremendously helpful during the testing of the developed algorithms. The diversity of instances available in Pabulib, varying in size, budget constraints – made it easy to systematically study real world low participation instances, which is the main topic of this work.

These libraries and databases offer a solid foundation, but neither currently focuses on robustness analysis or low-turnout modeling. Our project will address these gaps by extending the existing functionality with tools designed to evaluate the stability and sensitivity of outcomes.

1.5. Final Product Vision

The final product will be a well-documented Python library together with a set of reproducible examples and a theoretical part in the thesis. The goal of the software is to provide researchers and practitioners with practical, reliable tools to assess the stability of outcomes in approval-based multiwinner elections and PB instances, with particular attention to settings with low and potentially biased voter turnout. The emphasis of the project is on providing usable implementations and empirically useful tests and measures of the above mentioned election instances. Concretely, the project will consist of the following components:

- **A Python package.** A library that could be easily integrated with existing solutions and used in practice to conduct studies on real-world PB instances.
- **Robustness and stability measure.** Functions that estimate the probability that a given project is funded (local stability) and that the observed outcome set is reproduced (global stability) under resampled voting profiles. Exact-computation routines will be provided where efficient algorithms exist; in some cases, we provide proofs of computational hardness of studied problems.
- **Reproducible examples and benchmarks.** A collection of examples that demonstrate typical analysis on both synthetic instances and publicly available real-world PB datasets. Benchmarks will illustrate runtime and estimator variability for different instance sizes and voting rules.
- **Tests and documentation.** An automated test suite covering the correctness of core algorithms and invariants, plus API documentation and a short user guide that describes typical usage patterns and caveats (limitations of sampling, computational costs, and interpretation of probabilistic estimates).
- **Accompanying theoretical material.** The thesis will include the theoretical analysis that motivated algorithm design: complexity results and reductions for exact computation of selected robustness measures where relevant, and discussion of approximation/heuristic approaches in cases that are intractable.

Intended Impact. We hope that the final product will allow researchers and practitioners to systematically study how low or biased participation affects PB outcomes. It will help identify fragile instances that merit further outreach or procedural change, compare robustness across rules, and provide a reproducible foundation for empirical and theoretical follow-up work.

Chapter 2

Functional Scope

The library will be based on classes present in **pabutools**. We should extend some of the functionality present in this library as well as extend it with functions for robustness analysis.

2.1. Closer Look at Provided Functionality

Robustness analysis could be roughly divided into several types.

- Measures inspired by the work of Boehmer et al. [2025](#), namely the Cost-Reduction measure, Add-Approvals measure, Add-Singletons measure, and Rivalry-Reduction measure, as well as measures that are, in a sense, opposites of those introduced in the aforementioned article. The measures proposed in Boehmer et al. [2025](#) focus on the questions concerning how many voters must be persuaded to support a given candidate in order to change the outcome of an election. In contrast, we also consider measures intended to capture the extent to which the current winner could, for example, increase the cost of its project while still remaining the winner.
- We will also focus on analyzing how low participation in participatory budgeting affects the results. To do so, we will introduce new types of measures that allow us to estimate the level to which the originally cast votes are to be “believed”.
- Our library provides a visualization tool for analyzing Greedy-AV instances in practice, which we use to assess the performance of the SRM measure.
- Our project should include standard examples to help the end-user and it should be documented at the level that would allow for further development as well as improve ease of use.

2.2. Project Structure

As mentioned earlier, the final deliverable will be a Python library that builds upon existing tools such as `abcvoting` and `pabutools`, while also introducing new modules tailored to robustness analysis and participatory budgeting.

The library will be organized into several key components:

- **Core functionality module:** This module will implement or wrap standard voting rules (e.g., Plurality, Approval Voting) and PB algorithms, leveraging existing libraries wherever possible.

- **Robustness analysis module:** Will provide tools for estimating how susceptible election outcomes are to perturbations in the input, such as changes in voter preferences or additional participants. This includes both previously proposed measures (e.g., from the work of Boehmer et al. 2025) and novel ones aimed at capturing alternative notions of stability.

2.3. Functional Requirements

Below we state the functional requirements for the library using a (Must / Should / Could / Won't) style classification. Requirements are phrased conservatively: “Must” items describe the minimum deliverable set; “Should” items are desirable extensions that will be attempted if time allows; “Could” items are optional improvements for future work; “Won't” items explicitly limit scope for this project.

2.3.1. Must

- **Sampling robustness measures.** Implement functions that could be used to estimate or calculate precisely, whenever possible, robustness measures for single candidates as well as overall stability of election instances with low voter turnout.
 - In the case of what we called “Local Stability” in the introductory chapter, we will implement functions that estimate (or calculate directly) the probability that a given project/candidate is selected under resampling. Among those “Cost-Reduction”, “Perfect Approval Measure”, which are commonly used in practice.
 - In the case of what we called “Global Stability”, we will provide implementations of practical versions of algorithms designed to solve the problem studied in the second part of this thesis. While some of the algorithms that we describe run in polynomial time and compute the **Sampling Robustness Measure** accurately, they are not necessarily practical, as the ultimate answer in real world instances could be a really large integer, so instead to address practical concerns, we implement some heuristic approaches that could be used in practice.
- **Greedy-AV and Plurality voting rules.** Provide working, well-tested implementations or thin wrappers for those rules, that are commonly used in practice, which we shall use to analyze the effects of our work and perform robustness experiments. These implementations should be correct, documented, and usable as drop-in components for the simulation pipeline. (Where sensible, rely on `pabutools` code and provide adapters rather than re-implementing complex internals.)
- **Sampling / simulation engine.** Provide a Monte Carlo resampling engine that:
 - Build an empirical distribution from observed ballots.
 - Generates resampled electorates of arbitrary size n drawn i.i.d. from the empirical distribution.
 - Applies a chosen voting rule to each sample and collects outcome statistics (per-project selection probability, full-outcome frequency).

Estimators should expose parameters (number of trials, random seed) and return basic uncertainty information (e.g., standard error or confidence-interval approximation).

- **Minimal visualization utilities.** Provide simple, script-friendly plotting helpers to visualize simulation outputs (histograms of selection probabilities, bar charts of top projects, simple line plots of stability vs. turnout). Graphics may be implemented with `matplotlib` and must allow for a quick graphical estimation of results.
- **Integration with existing solutions.** Our implementation should be directly integrated with existing “pabutools” library.
- **Test suite.** Include unit tests (using `pytest`) for core logic (rules, sampling robustness measures, sampling engine).
- **PyPi integration.** Our implementation has to be integrated into Python packaging infrastructure. More concretely it should be available on PyPi, that would allow for the best end-user experience.
- **Documentation and usage examples.** Provide:
 - API docstrings for public functions.
 - A short user guide README demonstrating a typical workflow: load instance, run resampling, compute local/global stability, plot results.
 - Several reproducible examples (both real world instances as well as hand crafted scenarios).

2.3.2. Should

- **Budgeted PB support and cost handling.** Full support for variable project costs and budget constraints in the sampling and rule-application pipeline (beyond unit-cost special cases).
- **Tie-breaking options.** Expose and document tie-breaking strategies (deterministic lexicographic, uniformly random, fixed priority) and allow the user to select the tie-breaking mode for simulations.
- **Exact computation routines for small instances.** Where computationally feasible (small $n, |C|$), provide exact enumeration or dynamic-programming routines to compute selection probabilities exactly.
- **Reproducible benchmark scripts.** Add scripts that run timed benchmarks on the provided example instances and record runtimes / estimator variances for reference.

2.3.3. Could

- **Additional voting rules.** Add optional support or adapters for other rules (e.g., PAV, seq-PAV, other proportional approval rules) as time permits.
- **Integration with data repositories.** Helpers to fetch and normalize several public PB datasets for the reproducible examples.
- **Higher testing coverage and quality gates.** Aim to gradually increase test coverage and add static analysis (e.g., `flake8`, `mypy`) if convenient.

2.3.4. Won't

- **No exhaustive implementation of every multiwinner rule.** The project will not attempt to implement or fully optimize every known multiwinner or PB rule; instead it will prioritize a small, well-chosen set of rules sufficient for robustness analysis. Computationally heavy rules that require complex optimization (e.g., exact PAV winner computation for large instances) are out of scope unless provided by a robust external library.
- **No guarantees about representativeness beyond the model.** The software will implement the sampling model described in this thesis (resampling from the empirical ballot distribution) but will not attempt to correct for unknown sampling bias, perform causal inference, or provide guarantees that sampled outcomes reflect the true population beyond the assumptions stated in the thesis.
- **No production election administration features.** The library is not intended to be a production-grade election administration system (no voter authentication, no secure storage or audit trails, no ballot-casting frontend).
- **No hard performance scaling guarantees.** The project will not promise strict performance or scaling guarantees for arbitrarily large instances; performance optimizations will be opportunistic and documented as experimental.

2.3.5. Notes on Prioritization and Acceptance Criteria

- A requirement is considered *implemented* when:
 - The functionality is available through the public API with examples in the repository, and
 - Unit tests cover the core behaviors and pass on the main development environment.
- Items in **Must** form the minimum viable product (MVP) for the thesis software deliverable. **Should** items are high-priority extensions that will be implemented if schedule permits. **Could** items are optional and may be deferred to future work.
- Where the requirements call for probabilistic estimators (e.g., Monte Carlo), the library will expose the estimator configuration but will avoid hard numerical promises about error bounds in the API; users are referred to the documentation for recommended trial counts and interpretation guidance.

2.4. Quality Assurance

The primary goal of Quality Assurance (QA) is to ensure that the library is correct, reliable, and maintainable while remaining realistic about scope and guaranties. We will achieve that by implementing a number of techniques commonly used to ensure correctness of implementations in small scale projects.

2.4.1. QA Objectives

- Ensure functional correctness of core algorithms and adapters (Greedy-AV, Plurality wrappers, sampling engine, local/global robustness estimators).

- Detect regressions early and keep the public API stable for users of the thesis package.
- Make experiments and examples reproducible across development environments.
- Provide measurable acceptance criteria that determine when a requirement is considered implemented.

2.4.2. Testing Strategy

Testing is organized in several complementary layers. Each layer has concrete responsibilities and lightweight acceptance criteria.

Unit Tests.

- Scope: small, fast tests for individual functions and classes (rule wrappers, sampling primitives, probability estimators, utility functions).
- Framework: `pytest`.
- Acceptance criterion: all unit tests for **Must** functionality pass on the main development environment.

Integration Tests.

- Correctness of integration with `pabutools`, correct handling of provided functionality.
- Acceptance criterion: a set of representative end-to-end scenarios implemented as integration tests pass on CI.

Statistical Checks.

- Scope: lightweight tests that validate statistical behavior of estimators on small, analytically-understood instances (e.g., resampling from a degenerate distribution should produce near-deterministic outcomes).
- Strategy: compare the estimator output to the expected ranges and check that repeated runs with the same seed are identical.
- Note: statistical tests are necessarily heuristic — acceptance criteria are expressed as tolerance ranges.
- Statistical tests will also be implemented in the form of unit tests on real world instances, as well as hand-picked scenarios.

Performance and Scalability Tests.

- Some testing of real world instances to ensure that performance of selected measures and algorithms allows for them to be used in practice.

2.4.3. Test Data and Reproducibility

- Provide a small curated set of test instances.
- All tests that depend on randomness must set and document a fixed pseudorandom seed. Public examples and notebooks should specify seeds used to reproduce figures in the thesis.
- Reproducible environment specification will include a `pyproject.toml` with pinned versions for development. The repository README will document the recommended environment used for running tests and examples.

2.4.4. Code Quality and Static Analysis

- Enforce a consistent code style using a formatter (e.g., `black`) and a linter (e.g., `flake8`) as recommended development tools; these checks are run in CI.
- Optional lightweight type hints and periodic checks with `mypy` for public API modules; full type coverage is not required but encouraged for core modules.

2.4.5. Documentation and Examples Testing

- Keep the README and the short user guide up to date with the minimal workflow: load instance, run resampling, compute local/global stability, and generate a simple plot.
- Include a few reproducible examples.

2.4.6. Release and Packaging Tests

- Test the installation of the built package in a clean virtual environment as part of the release/packaging steps.
- PyPI publication is a **Must** deliverable; QA ensures that the packaged release installs and that core examples run after installation.
- We use `Poetry` to manage packaging and dependencies (the project is declared in `pyproject.toml` and locked with `poetry.lock`). Releases are built with `poetry build` and published to PyPI (via `poetry publish`); as part of QA we verify installation of the built wheel in a clean virtual environment and confirm that the core examples run after installation.

2.4.7. Acceptance Criteria and Metrics

- **Functional acceptance:** All **Must** features have unit or integration tests and those tests pass on the main development environment and CI.
- **Reproducibility acceptance:** The provided examples reproduce their documented outputs when run in the recommended environment with specified seeds.
- **Quality gates:** CI must pass linters/formatters and the test suite to be integrated in the project.

Chapter 3

Selected Implementation Aspects

This chapter focuses on practical implementation aspects of the developed software. We will explain the internal structure of the library, its main components, the applied technologies, and implementations of the most important algorithms and mechanisms used in the implementation will be presented in the next chapter. The chapter is intended both as a reference for further development and as an explanation of design choices made during the project.

3.1. High-Level Architecture and System Overview

The implemented solution is a Python library designed for robustness analysis of approval-based multiwinner elections and participatory budgeting (PB) instances.

3.1.1. Main Modules

The codebase is organized into the following logical modules:

- **core** - canonical election representations and wrappers around external libraries (`pabutools`, `abcvoting`); implementations of simple rules such as Plurality and Greedy-AV.
- **robustness** - local and global robustness measures, including Sampling Robustness Measure (SRM) and extensions of measures from the work of Boehmer et al. [2025](#).
- **visualization** - utilities for plotting measures of stability and visualizing results of Greedy-AV elections.
- **I/O** - wrapper functions to make the process of processing data easier.
- **tests** - unit and integration tests.

3.1.2. Conceptual Structure of the Library

Figure [3.1](#) presents the conceptual structure of our library.

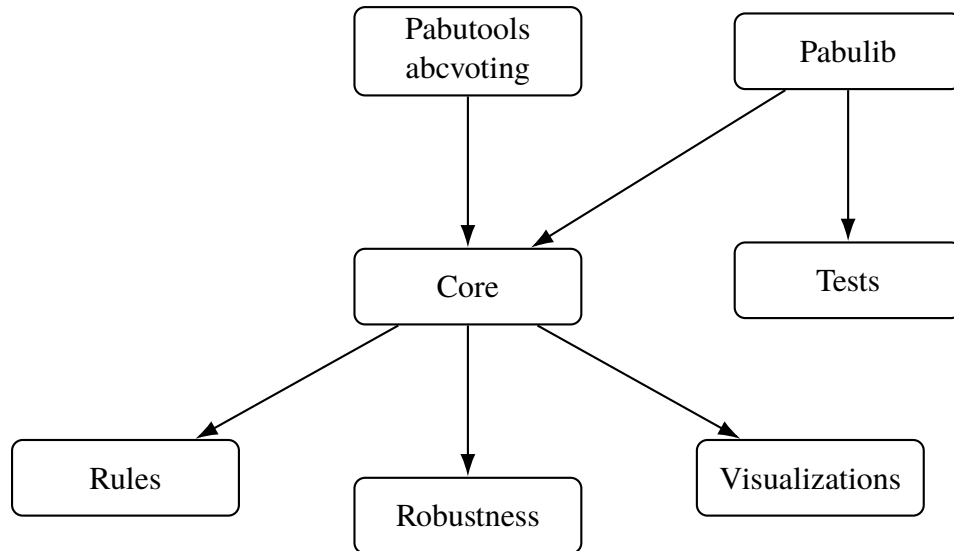


Figure 3.1: Conceptual module layout of the library and its dependencies.

3.2. Technology Stack

The following technologies were used in the implementation:

- **Python 3.10+** - main implementation language; chosen for ecosystem maturity and scientific computing support.
- **pabutools, abcvoting** - domain-specific libraries providing reference implementations of PB and approval-based voting rules.
- **pabulib** - real-world PB datasets used for testing and benchmarking.
- **NumPy, pandas** - efficient numerical operations and tabular data handling.
- **matplotlib** - reproducible, script-friendly visualization.
- **pytest** - unit and integration testing framework.
- **black, flake8** - code formatting and linting.
- **Poetry** - dependency management and PyPI packaging.
- **GitHub Actions** - continuous integration.

The stack was chosen to minimize the re-implementation of existing, well-tested functionality, while keeping the dependency footprint moderate and ensuring reproducibility.

3.3. Component Overview and Interfaces

3.3.1. Core Module

The `core` is a structural backbone of the whole project; it includes implementations of the Greedy-AV, Plurality rules, as well as helper functions that help us organize the I/O aspects of the project.

3.3.2. Robustness Module

This module implements both *local stability* (single-project performance) and *global stability* (outcome preservation) measures.

Implemented measures include:

- Cost-Reduction, Add-Approvals, Add-Singletons, Rivalry-Reduction Boehmer et al. [2025](#),
- Sampling Robustness Measure (SRM),

This is essentially the main component of the library, where all the algorithms are located.

3.3.3. Visualization Module

The `visualization` module provides lightweight helpers to generate the following:

- SRM measure plots
- Visualization of results Greedy-AV elections, they help us in our future analysis.

All plots are returned as `matplotlib` figures and can be exported to PDF or PNG.

3.4. Development Environment

A recommended development setup is as follows:

```
git clone https://github.com/yakimk/lib.git
cd lib
poetry install
poetry run pytest
```

The project is packaged using `Poetry` and distributed via `PyPI`.

3.5. Quality Assurance

Quality assurance is achieved through:

- **Unit tests** for core logic and algorithms,
- **Integration tests** on real-world PB instances from `pabulib`,
- **Continuous integration** enforcing test, lint, and format checks.

All stochastic tests use fixed seeds to ensure reproducibility.

3.6. Technical Challenges

The main technical challenges encountered include the following.

- handling tie-breaking nondeterminism in voting rules,
- scalability limits of exact computation,
- floating-point precision during SRM testing.

These challenges were addressed through explicit tie-breaking strategies, computing SRM measures exactly only for manageable instances, and applying computationally simpler measures of “local stability” in other places.

3.7. Summary

This chapter presented the technical realization of the software, its architecture, key modules, algorithms, and quality-assurance processes. We hope that the structure of our library is flexible and will allow for an easy integration with existing solutions as well as enable future developments.

Chapter 4

Work Organization

4.1. Project Characterization and Development Model

During the first semester, we focused on prototyping core ideas and properly understanding the goals of the current project; the second semester moved to theoretical work as well an incremental implementation of the developed algorithms and testing them on real-world instances. The main focus during the second part of the implementation was to:

1. prototype and validate algorithms on synthetic instances,
2. integrate with real-world datasets (`pabulib`),
3. implement robust API and testing infrastructure,
4. produce reproducible experiments and write-up.

4.2. People Involved

- **The Author** is responsible for writing this thesis, developing the accompanying Python library, and producing the theoretical results and algorithms described in Chapter 5.
- **Prof. dr hab. inż. Piotr Faliszewski** is the formal supervisor of the thesis. The Author is grateful for proposing the topic of the thesis and for providing valuable suggestions for its improvement.
- **Mgr inż. Łukasz Janeczko** acted as an informal supervisor of the project. The Author is grateful for numerous helpful discussions and suggestions regarding the content of the thesis.
- **Prof. Rafał Dreżewski** supervised the university project classes associated with this work. The Author had regular meetings with him to discuss the project's progress.
- **Dr. Dariusz Stolicki, Dr. Stanisław Szufa, Dr. Nimrod Talmon, Dr. Simon Rey, and others** contributed indirectly to this work as developers of the `Pabulib` and `Pabutools` libraries, which were extensively used by the Author.

4.3. Organization of Work and Tools

The project used the following process and tools:

- **Version control:** Git + GitHub.
- **Continuous integration:** GitHub Actions executing test suite, style checks, and build steps.
- **Communication:** periodic meetings with supervisors (weekly or biweekly), email for ad-hoc updates.

4.4. Engineering Practices

Practices applied during the project:

- **Unit testing and TDD where feasible:** core utilities were developed with unit tests; probabilistic components include deterministic test seeds.
- **CI-driven quality gates:** merges require passing tests and linters.

4.5. Timeline and Milestones

The high-level milestone schedule used during the project is summarized in Table 4.1.

Table 4.1: Project timeline (approximate).

Milestone	Period	Description / deliverables
Requirements & literature review	Feb – May	problem formulation, literature survey
Core prototypes	May – Jul	implementations of core utilities like Greedy-AV/Plurality implementation, basic visualizations and certain measures, small experiments and tests
Theoretical work	Aug – Sep	Most new results in the next chapter
Experiments and benchmarks	Oct – Dec	large-scale experiments, plots, metrics
Thesis writing	March - Jan	chapter drafts, polishing, final submission

4.6. Deliverables and Acceptance Criteria

The thesis deliverables and acceptance criteria were the following:

- **Software:** published Python package on PyPI, passing tests on CI.

- **Documentation:** README with usage examples and API documentation.
- **Experiments:** reproducible examples and plots that demonstrate the library functions on real-world instances.
- **Thesis:** theoretical analysis and empirical evaluation consistent with the code and experiments implemented.

4.7. Use of AI tools

In the preparation of this bachelor's thesis, AI-powered language tools were utilized to enhance the clarity, coherence, and overall quality of the written text, as well as helped to generate code that would create charts of tests. These tools assisted in tasks such as grammar correction, style refinement, and suggested phrasing. However, the content, ideas, research findings, and conclusions presented herein are solely the author's own. Any errors or omissions remain the responsibility of the author.

4.8. Concluding Remarks on Organization

The chosen iterative approach allowed the author to validate modeling choices early, mitigate algorithmic risks, and maintain a reproducible development and evaluation process that supports the research goals of the thesis.

Chapter 5

Project Results

We now come to the problem that is at the heart of this project, but before giving its description, let us first fix the notation that we use to describe election parameters:

- C - finite set of projects (candidates)
- V - finite set of voters
- $m = |P|$
- $n = |V|$
- $B \in \mathbb{R}_{\geq 0}$ - budget
- $c(\cdot) : C \rightarrow \mathbb{R}_{\geq 0}$ - cost function.

It is useful to additionally consider two more parameters that will be used to define the SRM measure (defined below):

- $N \in \mathbb{N}_+$ - sample size
- f - voting rule
- $K \subseteq P$ - set of projects selected by the rule f for election E , that is, $f(E)$

We consider the following counting problem: given a voting rule f , an election instance E , and a positive integer N , assuming that $K = f(E)$ is the set of winning projects, what is the number of unordered N -tuples of original votes (with repetitions allowed) on which rule f selects the same set of winning projects as in the original election (i.e., K is the winning set).

We give this problem the working name *Sampling Robustness Measure (SRM)*.

5.1. Theoretical Results

We present proofs of hardness of *Sampling Robustness Measure* even in very restricted cases of Greedy-AV (see Sections 5.1.1 and 5.1.2 for details), as well as effective algorithms for the case of Plurality in Subsection 2 of the current chapter.

SRM for Greedy-AV

First we present a (parsimonious, in fact one-to-one) reduction of #X3C to SRM for Greedy-AV with unit costs. In fact the result is a little stronger showing reduction to version of Greedy-AV, where each voter is allowed to approve of at most 3 candidates and in original result of the election every project except one is selected. Thus it can be interpreted as complexity of determining probability that a selected project will finish last in the election.

An instance of #X3C consists of a finite set X , where $|X| = 3k$ for some $k \geq 0$, and a family of subsets $S \subseteq \mathcal{P}(X)$, such that for each $S_i \in S$, $|S_i| = 3$. The problem is about finding number of k -element subsets of S that cover X . Note that we may assume that elements of S cover X , i.e. that $\bigcup S = X$. This problem is known to be #P-complete.

Theorem 5.1.1. *The SRM for Greedy-AV with unit costs is #P-complete, even when each voter approves of precisely three candidates and in the original election every project except one is selected.*

Proof. We reduce #X3C to SRM for Greedy-AV.

Suppose we are given #X3C instance with a notation as above. We construct an election as follows:

- Set of candidates $C = X \cup \{l\}$, where l is some additional project that will be the only project that is not selected. Each project will have unit cost.
- Set of voters $V = S$, meaning one voter for each element of S . Voter which corresponds to an element $S_i \in S$ will be set to approve of candidates that correspond to elements of S_i .
- Budget $B = 3k = |X|$.
- Additionally we setup the tie-breaking by considering some strict linear order ($<$) on C and require that l be a greatest element of this order.

Given such an election instance it is clear that every project except l has some supporters, this is precisely because we require $\bigcup S = X$ in #X3C instance. It is also clear that since every project has unit cost and $B = |X|$ every project except l will be selected ($K = C \setminus \{l\}$).

By using constructed election and taking sample size $N = k$ we get an instance of SRM for Greedy-AV. Moreover since l wins every tie-breaking by construction, every sample of k voters that preserves the result of the election, must include supporters of every project except l and thus the corresponding family of k elements of S must cover X . Clearly the opposite is also true, i.e. every cover of X by k elements of S produces a sample of k votes such that every project except l is selected. □

As noted before, in our construction we only used voters who approve of at most three candidates and the set of candidates selected in the first place (K) was all but one project. That allows us to interpret the result of the above theorem as saying that even computing the number of ways for a single project to be in the last place in a given election instance is (in general) hard.

By carefully selecting the problem to reduce from, we could achieve even slightly stronger results. In particular, we can achieve not only a better (and best possible) bound on the number of candidates that a voter approves of but also (simultaneously) bound the number of voters that approve of a given candidate.

Before stating the result, we introduce a simple notion of a **(hyper)graph of an election instance (for approval ballots)**, which is a hypergraph, whose vertices are the set of candidates

and each (hyper)edge corresponds to a voter and connects all the vertices corresponding to candidates that this voter approves of. In particular if we restrict to 2-Approval instances (that is, Greedy-AV instance, where each voter is allowed to approve of no more than 2 candidates), the hypergraph of such instances will in fact be a graph.

Theorem 5.1.2. *SRM for Greedy-AV with unit costs is #P-complete, even when each voter approves of exactly two candidates, in the original election every project except one is selected, every (selected) candidate has exactly three supporters and selected candidates can be divided into two disjoint groups, such that each voter approves of precisely one candidate in each group.*

Proof. This result is a straightforward reduction from #PerfectMatchings for bipartite 3-regular graphs, which is known to be #P-complete (see, for example, for reference). The reduction can be seen as very much similar to the one used in the proof of the previous theorem (in particular if we interpret #X3C as #PerfectMatching for a graph whose vertices are elements of the set that we cover and whose (hyper)edges are 3-element covering sets, both constructions can be seen to be identical).

To reduce #PerfectMatching for bipartite 3-regular graphs, it is enough to consider an election instance (with Greedy-AV rule) of a given graph, that is vertices become candidates and each edge is considered as a voter who approves of precisely both of the candidates corresponding to the vertices it connects.

If we add another voter l and set it as always winning in tie-breaking and set the number of samples N to $|V|/2$ (half the number of vertices). As in the previous proof, setting the budget to the number of candidates minus one allows us to ensure that every N samples of voters in this election that preserve the outcome correspond bijectively to perfect matchings in the graph of this election, which is precisely our original graph (without the vertex corresponding to the voter l).

□

In particular, the previous result implies hardness of SRM for so-called k -Approval, which is a rule like Greedy-AV, except each voter is allowed to approve of at most k candidates.

Corollary 5.1.2.1. *SRM for k -Approval is #P-complete for $k \geq 2$, even with some additional assumptions as in the previous theorem.*

In fact, a slight variation of the problem we reduce from gives us a different result still. To ensure that #PerfectMatching remains #P-complete for a given class of graphs, they have to be non-planar, because otherwise there exist algorithms to count the number of perfect matchings in polynomial time (see the works of Kasteleyn 1987, Temperley et al. 1961, Kasteleyn 1963). Unfortunately, as far as we can tell, the condition for a graph of an election being planar is not a natural condition to impose on an election instance, but nonetheless we can achieve hardness results for election instances with an election graph being planar and 3-regular, by choosing the right problem to reduce from.

Theorem 5.1.3. *SRM for Greedy-AV with unit costs is #P-complete, even when each voter approves of exactly two candidates, in the original election every project except one is selected, the corresponding graph of an election is planar, and each (selected) candidate is approved by precisely three voters.*

Proof. In this proof we reduce from a problem of counting the number of edge covers (that is, the number of subsets of the set of edges of a graph such that each vertex is connected to some

edge in a chosen subset) of 3-regular planar graphs, which is known to be #P-complete (see the work of Cai et al. 2012).

It may be quite surprising that we are able to reduce a problem of counting all edge covers (without replacement) to a problem of counting the number of possible vote samples (with replacement). The key step in the proof is to reduce the problem of finding the number of edge covers to a problem of finding the number of edge covers of some fixed size N with multiplicity (we call them multicovers of size N), meaning that we consider all unordered sequences of edges that form an edge cover. In particular, we allow repeating edges.

To construct this reduction, we have to come up with a technique of calculating the number of edge covers from the number of edge covers with multiplicities.

First, we will look at multiset edge covers of size $N = |E|$, equal to the number of edges in the starting graph. Now let c_k denote the count of edge covers (not multicovers) with k distinct edges. We have

$$\#EdgeCovers = \sum_{k=1}^{|E|} c_k.$$

We argue that

$$\#MultisetCovers = \sum_{k=1}^{N=|E|} \binom{N-1}{k-1} \cdot c_k.$$

It is not hard to see this, since once we fix k distinct elements of the cover, it remains to count the possible multiplicities. This is done precisely by the factor $\binom{N-1}{k-1}$, which can be interpreted as the number of positive k -element sequences (tuples) of numbers that sum up to N .

Using the above equations, we can see that reducing the problem of counting edge covers to counting multicovers of size $|E|$ can be expressed as a problem of finding these coefficients c_k , once we know the number of $\#MultisetCovers$. That being said, computing c_k can be done, for example, by computing the inverse of the matrix

$$M = \begin{pmatrix} \binom{0}{0} & \cdots & \binom{|E|-1}{0} \\ \vdots & \ddots & \vdots \\ \binom{0}{|E|-1} & \cdots & \binom{|E|-1}{|E|-1} \end{pmatrix}.$$

For further information consult the survey of Boyadzhiev 2018.

Essentially, this finishes the proof, since computing size $N = |E|$ edge multicovers of a given graph can be reduced (in the same way as before) to the problem of calculating SRM for Greedy-AV for the corresponding graph of this election instance. Taking our original assumptions into account (the graph being 3-regular and planar), we can see that this proves the claim. $\square$

As was noted above, we do not have a natural interpretation for the condition that a graph of an election is planar. Still, it is quite interesting to see that computing SRM for Greedy-AV is a problem that has many natural interpretations, and using it we can produce simple reductions from a range of graph problems that do not reduce naturally to one another.

We did not pay much attention so far to fixing the number of samples N of an instance of SRM for Greedy-AV. In fact, our first proofs showing #P-completeness used reductions to instances with a smaller sample size in comparison to the overall number of votes in the original election. Using the technique from a previous theorem, we can now also confirm that SRM for Greedy-AV remains hard even when the number of samples equals the number of votes.

Corollary 5.1.3.1. *SRM for Greedy-AV with unit costs is #P-complete even when we assume $N = n$ (i.e., sample size equals the number of votes).*

Proof. The exact same technique that was used to reduce from counting the number of edge covers of a graph to counting the number of size $|E|$ edge multicovers of a graph in the previous proof can be used to modify our first proof in this section (reduction from #X3C to SRM for Greedy-AV) to instances with sample size equal to the number of votes in the original election. $\square$

Plurality

As we have seen, SRM for Greedy-AV (in fact, already for k -Approval) even with unit costs is #P-complete. We now consider the *Plurality* rule, which is a version of Greedy-AV where each voter is allowed to approve exactly one candidate.

As expected, SRM for this voting rule is easier than in the case of Greedy-AV.

We start by investigating the simplest possible case, that is, SRM for Plurality with exactly one supporter for each project and unit costs. We also ignore tie-breaking for now.

Lemma 5.1.4. *SRM for Plurality with unit costs, where each candidate has exactly one supporter and each winning project receives a strictly higher score in the voting than any losing project, is in FP, and the polynomial-time algorithm depends only on the budget, number of samples, and number of projects.*

Proof. As before, denote by C the set of all projects and by K the set of projects that win the initial election.

Since we are dealing with unit costs and, by assumption, each project chosen in the original instance of an election receives a strictly higher score than any losing project, we can split the problem by some parameter, call it M , that will be the maximum among all scores that losing projects receive.

The idea is simple: we first fix M and then add $M + 1$ votes to projects in K . The remaining votes will be distributed partly among winning projects and partly among losing projects, ensuring that at least one losing project actually receives M votes and no losing project receives more than M votes. So we can additionally fix T , the number of approvals distributed among losing projects (note that it makes sense to consider only $M \leq T \leq (\text{number of losing projects}) \cdot M$, but we will not worry about efficiency).

Ultimately, the number of valid samples can be written as

$$\sum_{M: 0 \leq M \leq \lfloor m/k \rfloor - 1} \sum_{T=M}^{\min\{rM, m-k(M+1)\}} f(m - k(M+1) - T, k) \cdot g(T, M, r), \quad (1)$$

where

$$f(R, k) = \binom{R + k - 1}{k - 1}$$

counts ways to allocate the extra R approvals among k winners (after giving each winner the baseline $M + 1$), and

$$g(T, M, r)$$

is the number of r -tuples $(x_1, \dots, x_r)$ with $0 \leq x_i \leq M$, $\sum_i x_i = T$, and at least one $x_i = M$.

Again, to reiterate, $f(x, y)$ represents the number of ways of distributing x votes among y winning projects. The distribution of votes among winning projects provides no further constraints on the way we must distribute the votes. This means that each y -tuple of nonnegative integers such that its elements sum to x is a valid distribution of votes. There are precisely $\binom{x+y-1}{y-1}$ such tuples. This shows that computing $f(x, y)$ is easy, and we can focus on $g(x, y, z)$.

The function $g(x, y, z)$ represents the number of ways to assign x elements to z projects, provided that at least one project has exactly y supporters and each project receives no more than y supporters. We can interpret this number as the number of ordered z -tuples of nonnegative integers such that each tuple sums up to x , there exists an element of the tuple equal to y , and each element is at most y . This function can be computed using standard dynamic programming techniques. We present one simple approach in an algorithm below.

Algorithm 1: COMPUTE_G_NAIVE(T, M, r) — naive DP (single-supporter case)

Input : integers $T \geq 0, M \geq 0, r \geq 0$
Output $g[t] = g(t, M, r)$

```

1  initialize arrays  $dp\_0[0..T] \leftarrow 0, dp\_1[0..T] \leftarrow 0$ ;
2   $dp\_0[0] \leftarrow 1$ ;
3  for  $i \leftarrow 1$  to  $r$  do
4      initialize  $new\_dp\_0[0..T] \leftarrow 0, new\_dp\_1[0..T] \leftarrow 0$ ;
5      for  $s \leftarrow 0$  to  $T$  do
6          if  $dp\_0[s] > 0$  then
7              for  $c \leftarrow 0$  to  $\min(M, T - s)$  do
8                  if  $c < M$  then
9                       $new\_dp\_0[s + c] += dp\_0[s]$ 
10                 end
11             else
12                  $new\_dp\_1[s + c] += dp\_0[s]$ 
13             end
14         end
15     end
16     if  $dp\_1[s] > 0$  then
17         for  $c \leftarrow 0$  to  $\min(M, T - s)$  do
18              $new\_dp\_1[s + c] += dp\_1[s]$ ;
19         end
20     end
21 end
22  $dp\_0 \leftarrow new\_dp\_0, dp\_1 \leftarrow new\_dp\_1$ ;
23 end
24 return  $dp\_1[T]$ 

```

As both f and g are computed in polynomial time with respect to the number of samples and projects, this finishes the proof of the lemma. $\square$

We now improve the algorithm from the previous lemma to include situations where each project is allowed to have an arbitrary number of supporters.

Lemma 5.1.5. *SRM for Plurality with unit costs, where each winning project receives a strictly higher score in the voting than any losing project, is in FP, and the polynomial-time algorithm depends only on the budget, number of samples, and number of projects.*

Proof. We use the same notation as in the proof of the previous lemma. We will use the same approach as before, except now we have to deal with multiplicities of supporters, luckily it is not too hard, given the dynamic algorithm from the previous lemma:

Algorithm 2: COMPUTE_G_WEIGHTED(T, M, r, s_list) — naive DP with multiplicities (multi-supporter)

Input : integers $T \geq 0, M \geq 0, r \geq 0$; list $s_list[1..r]$ = number of supporters per processed loser (order corresponds to the sequence of losers processed)
Output $g[T]$ - number of assignments $(x_1, \dots, x_r)$ with $0 \leq x_i \leq M, \sum x_i = t$, and at least one $x_i = M$, counting supporter-level multiplicities

1 *Preliminaries: compute per-project weight arrays*
 2 For each loser index $j = 1..r$ compute $ways_j[0..\min(M, T)]$ where

$$ways_j[c] = \binom{s_list[j] + c - 1}{c}$$

(you may compute iteratively: $ways[0] = 1, ways[c] = ways[c - 1] \cdot \frac{s+c-1}{c}$);

```

3 initialize arrays  $dp\_0[0..T] \leftarrow 0, dp\_1[0..T] \leftarrow 0$ ;
4  $dp\_0[0] \leftarrow 1$ ;
5 for  $j \leftarrow 1$  to  $r$  do
6   initialize  $new\_dp\_0[0..T] \leftarrow 0, new\_dp\_1[0..T] \leftarrow 0$ ;
7   for  $s \leftarrow 0$  to  $T$  do
8     if  $dp\_0[s] > 0$  then
9        $v0 \leftarrow dp\_0[s]$ ;
10       $maxc \leftarrow \min(M, T - s)$ ;
11      for  $c \leftarrow 0$  to  $maxc$  do
12        if  $c < M$  then
13           $new\_dp\_0[s + c] += v0 \times ways_j[c]$ ;
14        end
15      else
16         $new\_dp\_1[s + c] += v0 \times ways_j[c]$ 
17      end
18    end
19  end
20  if  $dp\_1[s] > 0$  then
21     $v1 \leftarrow dp\_1[s]$ ;
22     $maxc \leftarrow \min(M, T - s)$ ;
23    for  $c \leftarrow 0$  to  $maxc$  do
24       $new\_dp\_1[s + c] += v1 \times ways_j[c]$ ;
25    end
26  end
27 end
28  $dp\_0 \leftarrow new\_dp\_0, dp\_1 \leftarrow new\_dp\_1$ ;
29 end
30 return  $dp_1[T]$ 
```

Essentially, what we do is add a weight factor of $ways_j[c]$ to each DP table update. At the same time it is obvious that approach that would have to include the tie-breaking rules into

account, would have to be quite different. □

While the algorithm from the previous lemma is quite elementary, at the same time it is obvious that approach that would have to include the tie-breaking rules into account, would have to be quite different. One approach would be to treat the cases, when one of the losing projects has the same number of supporters as one of the winners, separately. It surely plays well, with what we've done so far, the two cases are disjoint and could be computed essentially separately.

5.2. Practical Experiments

Before showing the results of our study, let us introduce some simple tool for visualizing Greedy-AV election, that will be helpful in analyzing the results and usefulness of our work.

5.2.1. Greedy-AV Outcome Visualization

We will use the following simple visualization of the Greedy-AV (Sequential Approval Voting) outcome in our later analysis. For a given election instance, we draw a scatter plot in which:

- The **x-axis** represents the number of supporters (approval score) of each project.
- The **y-axis** represents the cost of each project.
- Each project is plotted as a point at coordinates (supporters, cost).
- Points corresponding to projects selected by Greedy-AV are colored in **blue**, and non-selected projects in **red**.

Additionally, we draw a piecewise-constant and piecewise-linear curve showing the **remaining budget** during the execution of Greedy-AV:

- At each step of the algorithm, before considering the next project in the decreasing-support order, a horizontal line is drawn at the level of the current remaining budget.
- When a project is selected, the remaining budget drops by its cost, which is visualized as a vertical segment connecting the previous and the new budget levels.

The resulting diagram makes it easy to identify which projects can be afforded at each stage, and in particular:

- Every project whose cost lies *strictly below* the budget curve at its support level is selected by Greedy-AV.
- Horizontal or vertical projections from a point to the budget curve correspond to various approval- or cost-based quantities that will be useful in our later analysis.

It is best to follow-up with an example. This and most of the following examples are based on the real life data and are taken from the Pabulib library (see the work of Stolicki et al. [2020](#)).

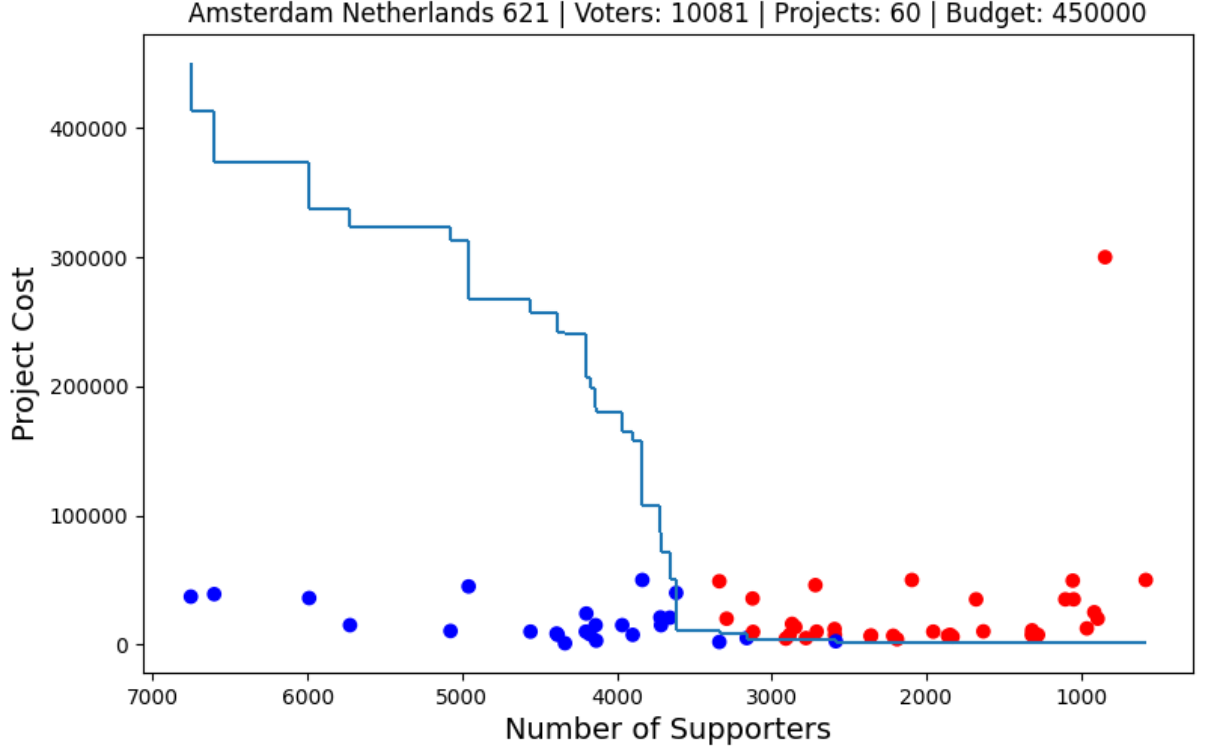


Figure 5.1: Example of a visualization of Greedy-AV instance.

This type of visualization will be convenient for our further analysis. We can clearly see the relationship between costs, approvals and the remaining budget at each step: every project lying strictly below the budget curve is selected by Greedy-AV, and simple geometric projections from a point to the curve correspond to useful quantities.

Formally, let a project be represented by the point $p = (s, c)$, where s is its number of supporters (approval score) and c its cost, and let $\Gamma = \{(s, B(s))\}$ denote the piecewise-constant / piecewise-linear remaining-budget curve produced by Greedy-AV. The visualization yields several immediately interpretable projections and distances:

- **Perfect cost.** The *vertical* projection of p onto Γ , $p_{\text{cost}} = (s, B(s))$, gives the budget available at the project's support level. The vertical gap $c - B(s)$ (positive or negative) measures how much the project's cost must change (decrease or could be considered “surplus” if negative) in order to be exactly on the selection threshold at that support level.
- **Perfect approval.** The *horizontal* projection of p onto Γ , $p_{\text{app}} = (s', c)$ with s' such that $(s', c) \in \Gamma$ (when defined), gives the approval level at which the project's cost would be exactly affordable. The horizontal gap $|s' - s|$ measures how many additional (or fewer) supporters would be required for affordability at the current cost.
- **Cost function.** After choosing a cost function c for a selected project, that would determine the approximate “cost” of getting n new supporters, we could find an *optimal point* on the curve with respect to c

$$\text{opt}(p) = \arg \min_{q \in \Gamma} c_p(q).$$

This point is the closest point on the budget curve to the project under the chosen notion of distance. Because the choice of metric involves a scaling between cost and supporters,

this projection is inherently *subjective* — it depends on how much, e.g., one extra voter is considered to be worth in units of cost.

All of these constructions are computationally cheap for the piecewise curve Γ produced by our visualization for Greedy-AV instance. This suggests several approaches one might take to study stability of elections in this setting, but we will focus on concrete applications of when it comes to analyzing the results of the Sampling Robustness Measure.

This visualization could be also used to analyze larger instances below, we give an example of such instance (we remove the labels, for the purpose of readability).

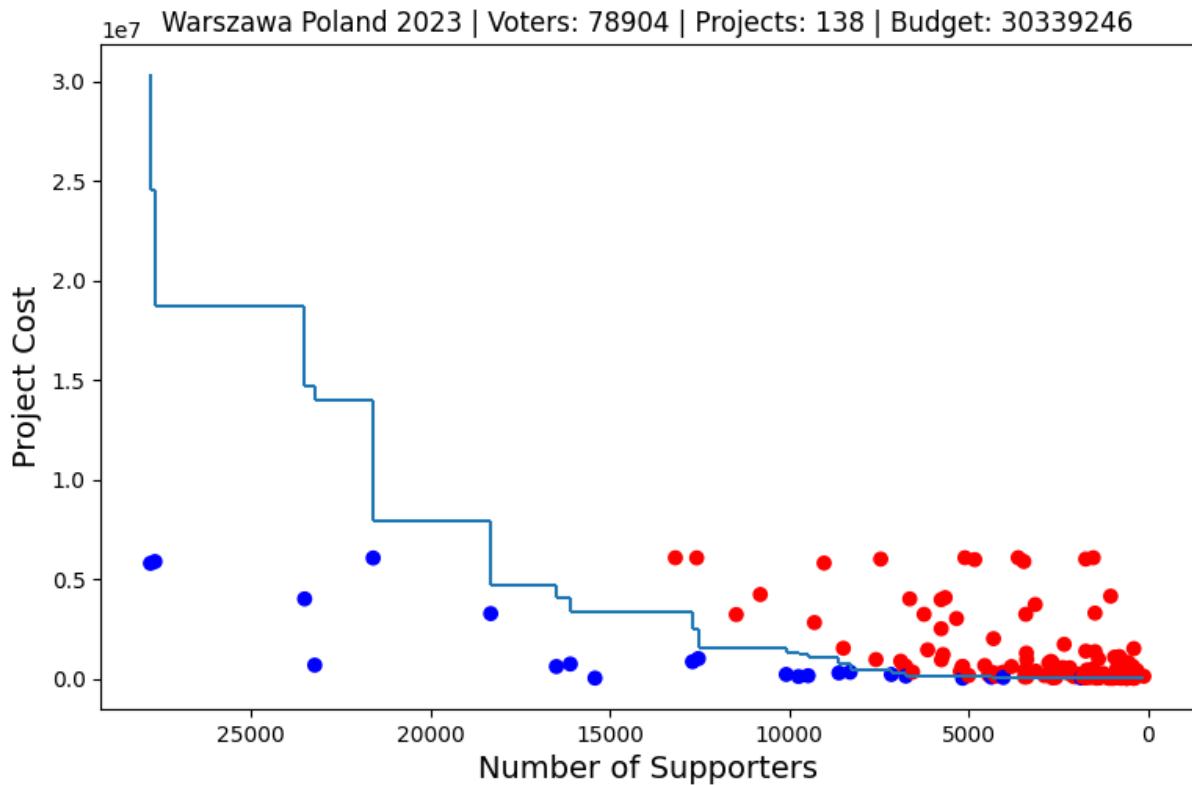


Figure 5.2: Example of a visualization of a larger Greedy-AV instance.

5.2.2. Analysis of SRM for Real-Life Instances

To demonstrate the practical usefulness of our method, we analyze three instances from the Pabulib library(see the work of Stolicki et al. [2020](#)):

- **Toulouse, France 2022**, Mirail-Université Reynerie-Bellefontaine,
- **Assen, Netherlands 2024**,
- **Worldwide Mechanical Turk: Knapsack 6**.

These instances were chosen because they exhibit markedly different behaviors under the SRM measure, and their Greedy-AV visualizations allow us to interpret these differences.

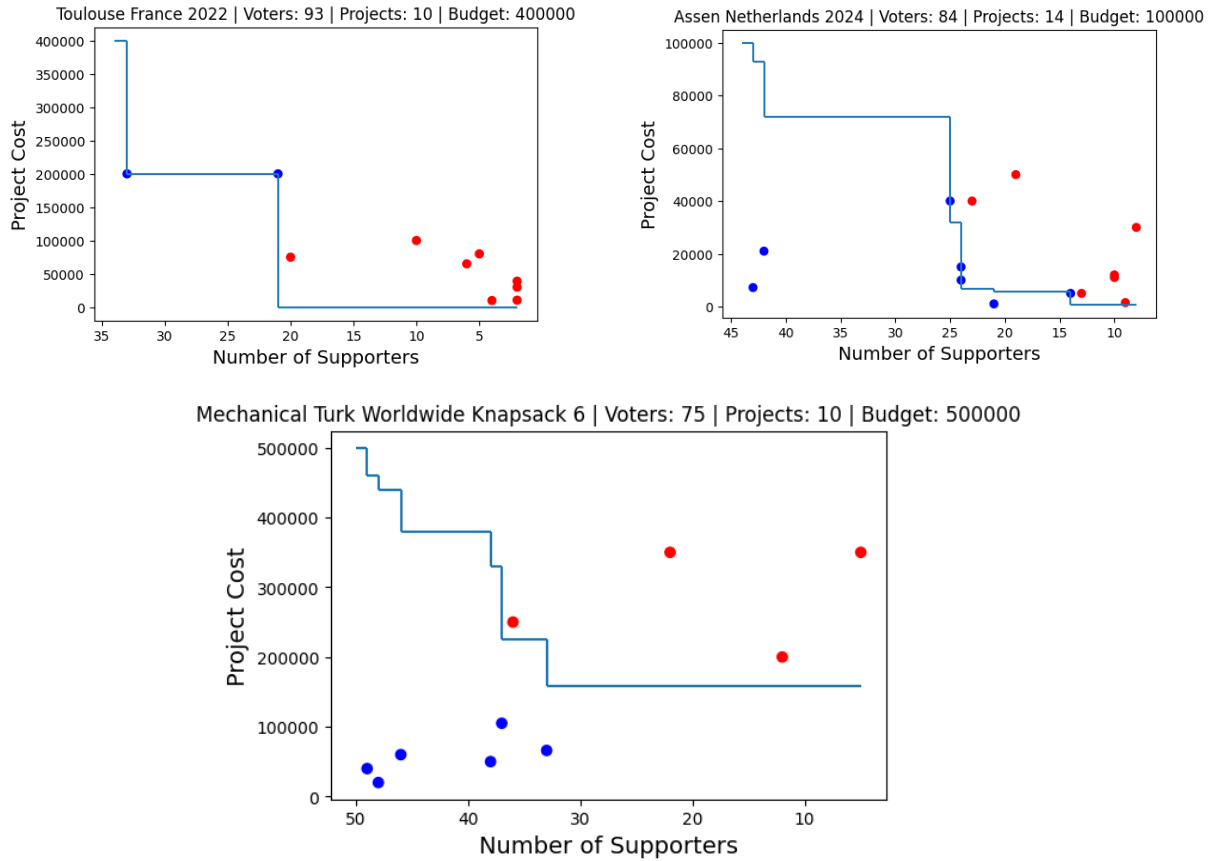


Figure 5.3: Visualization of Greedy-AV on three small instances.

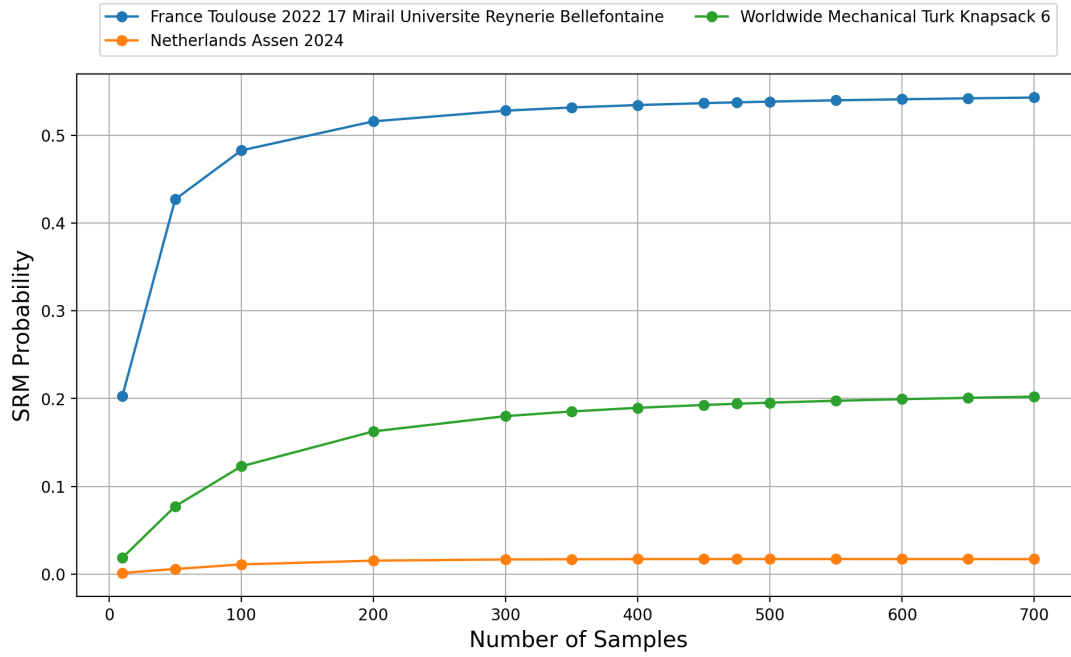
Let us now quickly analyze the results.

One distinctive feature of the Toulouse instance is that it contains two “dominant” winning projects, while the remaining projects—although cheaper—receive substantially fewer approvals.

The Assen instance is particularly interesting: it contains two pairs of rival projects, $(8, 1)$ and $(14, 5)$, where one project in each pair is selected, and the other is not. Since these pairs lie very close in the (support, cost) plane, this instance is ideal for testing whether SRM detects sensitivity to small perturbations.

The Knapsack instance (as its name suggests) is designed for testing knapsack solvers. It has the property that each selected project strictly dominates each non-selected project both in support and cost. This produces a very “stable” Greedy-AV outcome.

After examining the visual structure of these instances, we now turn to the results of the sampling tests.

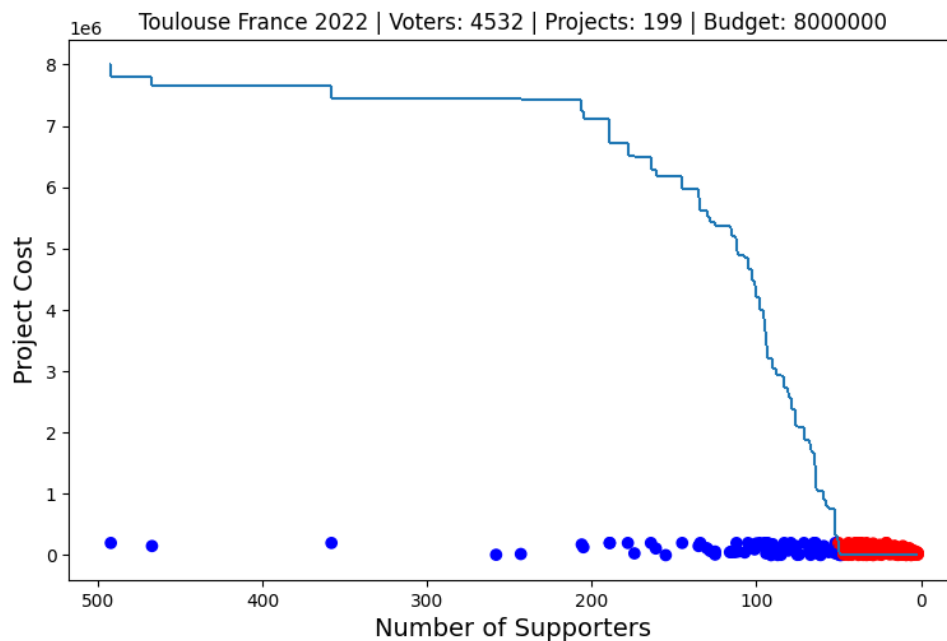


As we can see, the highest SRM score is achieved by the Toulouse instance, followed by the Knapsack instance, while the Assen instance obtains the lowest score.

The low SRM value for Assen is expected: as indicated by the visualization, its outcome is sensitive to small changes in support or cost, since several losing projects lie extremely close to winning ones. A small amount of added support or reduced cost can therefore flip the election outcome.

5.2.3. Larger Instances

Let us now consider larger election instances. For example, in the following case we obtain the Greedy-AV visualization shown below.



Running the SRM measure on this instance gives the following results:

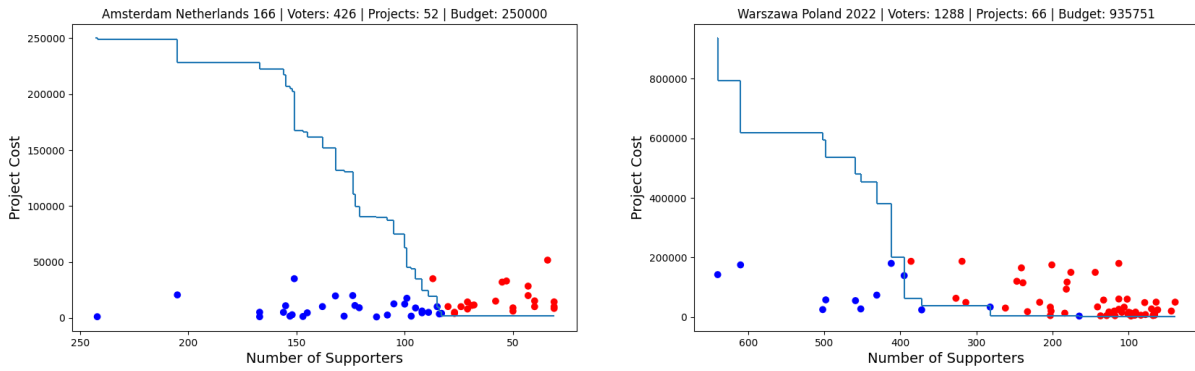
Table 5.1: SRM results for a larger instance

Instance	100	200	300	400	1000
Toulouse	8.65×10^{-37}	2.83×10^{-30}	5.37×10^{-26}	1.15×10^{-22}	1.23×10^{-16}

Note: Sample sizes 10 and 50 are omitted as they were approximately 0.

One immediate observation is that SRM values for large instances tend to be much lower than for smaller ones. This is natural: when projects are numerous and distributed across a wide range of costs and approvals, the last winners are typically surrounded by many alternative projects. Thus, many small perturbations can change the outcome.

To compare across different large instances, we examine their corresponding election graphs:



We then compute the SRM values:

Table 5.2: SRM results for several large instances

Instance	100	200	300	400	1000
Toulouse	8.65×10^{-37}	2.83×10^{-30}	5.37×10^{-26}	1.15×10^{-22}	1.23×10^{-16}
Amsterdam	1.27×10^{-9}	5.36×10^{-8}	7.21×10^{-7}	2.86×10^{-6}	1.26×10^{-4}
Rembertów	5.59×10^{-7}	7.11×10^{-6}	1.77×10^{-5}	2.49×10^{-5}	1.00×10^{-5}

Note: Sample sizes 10 and 50 are omitted as they were approximately 0.

Two observations stand out:

1. Although all SRM values are small, they stabilize as the sample size increases.
2. There are substantial differences across instances. In particular, the Toulouse instance consistently exhibits the lowest SRM scores.

Inspecting its election graph reveals a plausible explanation: the Toulouse instance contains a large cluster of losing projects located very close to the point where the remaining budget becomes insufficient. In contrast, the other instances are more evenly distributed, with no comparable cluster near the cutoff.

This suggests that one structural feature detected by SRM is the **density of near-winning losing projects**: if many projects lie just above the affordability threshold, small random perturbations are more likely to change the set of winners. This aligns well with the observed behavior.

5.2.4. Comparison of SRM Measure of Elections in Poland and Worldwide.

We continue our analysis by comparing the SRM measures of the elections held in Poland with those held in other parts of the world. As an example we take elections held in Łódź and Amsterdam and Budapest. Below we present the Greedy-AV visualization of the studied instances.

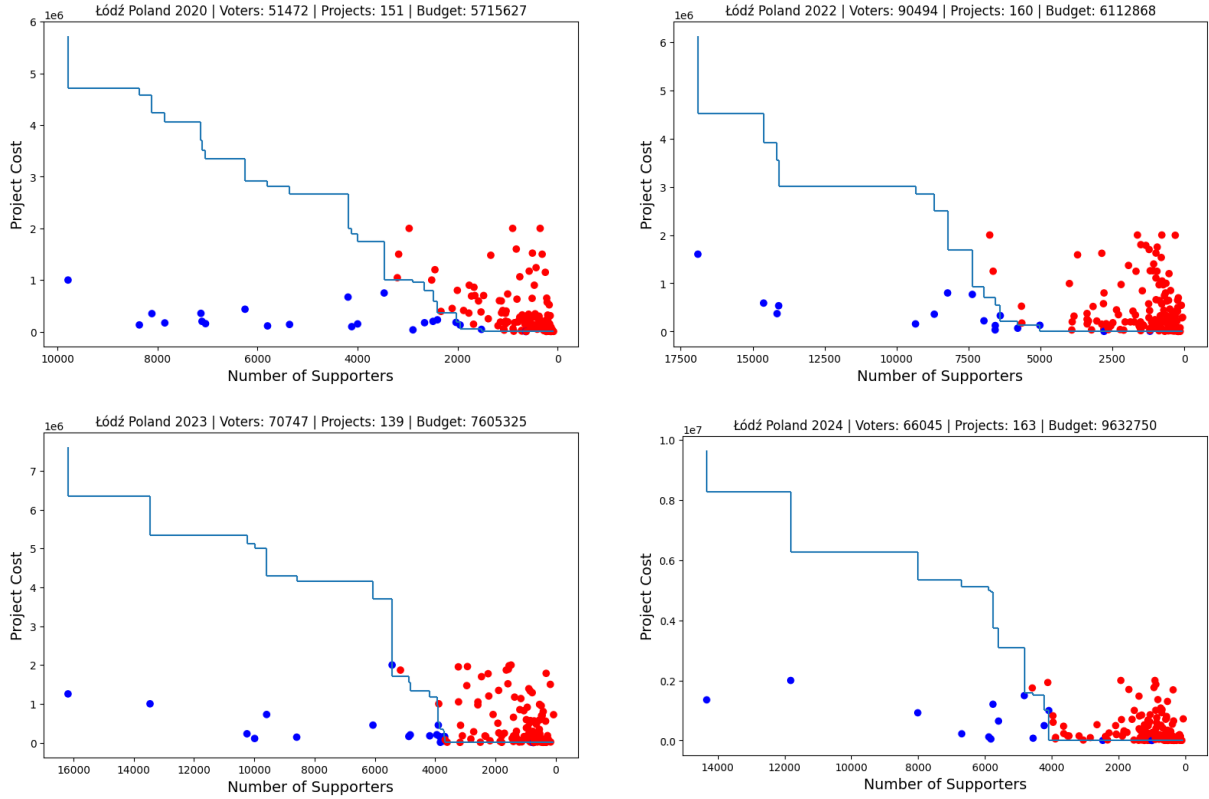


Figure 5.4: Visualization of Greedy-AV of four elections in Łódź.

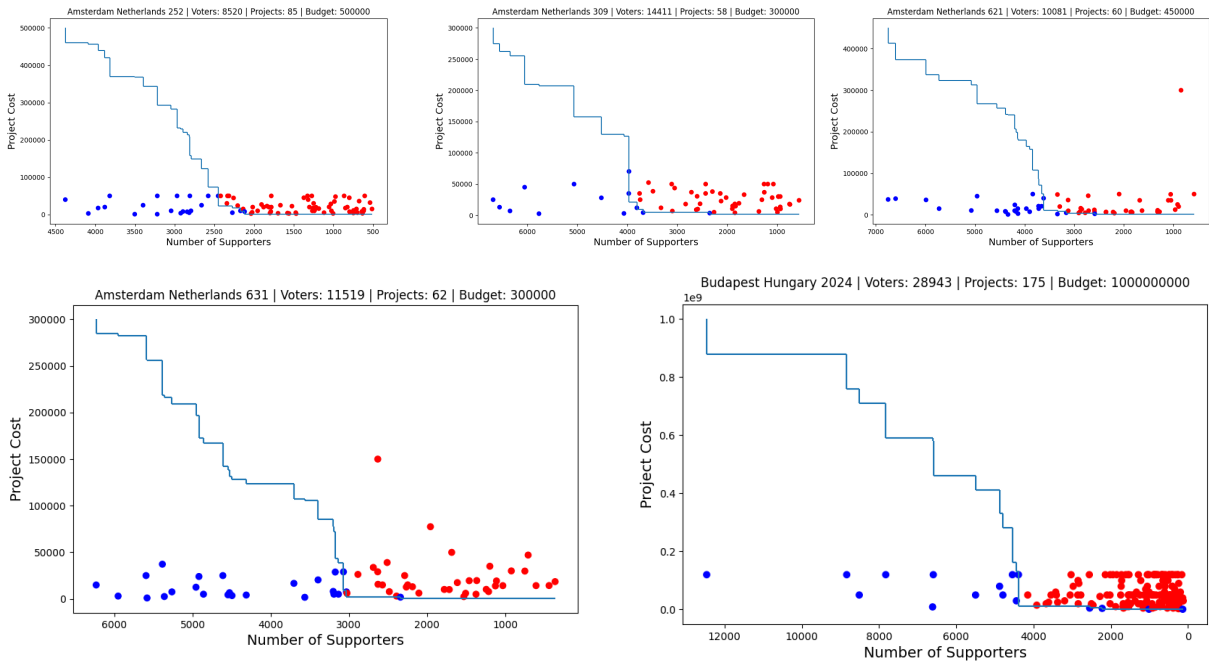


Figure 5.5: Visualization of Greedy-AV of five elections held in Amsterdam and Budapest.

While these instances differ in their general properties like budget, number of projects and voters, we still observe a few distinctive features of resulting SRM graphs.

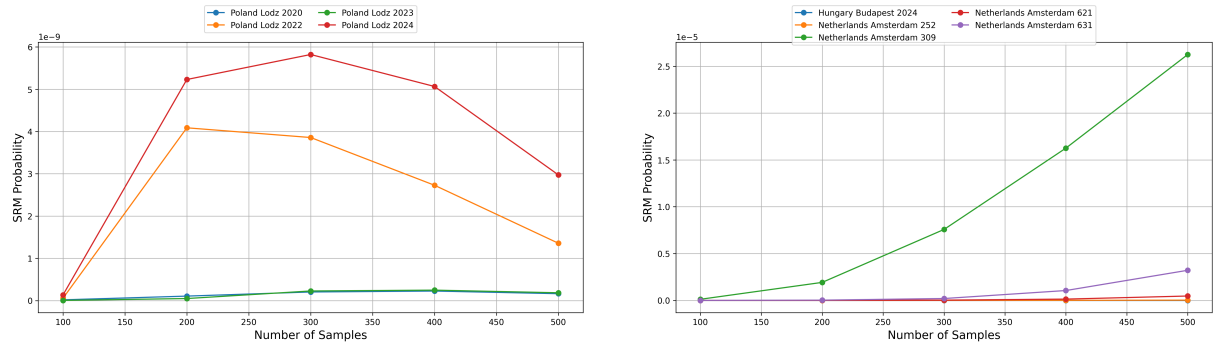


Figure 5.6: Comparison of SRM graphs for elections in Poland (Łódź) and worldwide (Amsterdam and Budapest).

One interesting pattern that we notice is that elections held in Łódź are the only ones among those tested that show the consistent decreasing of SRM score as the number of samples is increased. While it is hard to give a direct explanation of this phenomenon it may suggest that results of the elections held there were fundamentally unstable and there was a lot of rivalry among the projects that did get elected and those that didn't.

On the other hand, SRM scores of elections held in Amsterdam in Budapest (especially in the Amsterdam 329 instance) seem to grow quite rapidly with respect to the number of samples computed, which may suggest that the results were indeed representative of the population needs and would not change much if more people voted.

Bibliography

- Boehmer, N., P. Faliszewski, Ł. Janeczko, D. Peters, G. Pierczyński, Š. Schierreich, P. Skowron, and S. Szufa (2025). *Evaluation of Project Performance in Participatory Budgeting*. arXiv: 2312.14723 [cs.GT]. URL: <https://arxiv.org/abs/2312.14723>.
- Boyadzhiev, K. N. (2018). *Notes on the Binomial Transform*. World Scientific.
- Cai, J.-Y., P. Lu, and M. Xia (2012). “Holographic Reduction, Interpolation and Hardness”. In: *Computational Complexity* 21.4, pp. 573–604. DOI: 10.1007/s00037-012-0044-6. URL: <https://doi.org/10.1007/s00037-012-0044-6>.
- Kasteleyn, P. W. (Feb. 1963). “Dimer Statistics and Phase Transitions”. In: *Journal of Mathematical Physics* 4.2, pp. 287–293. DOI: 10.1063/1.1703953. eprint: https://pubs.aip.org/aip/jmp/article-pdf/4/2/287/19156929/287_1_online.pdf. URL: <https://doi.org/10.1063/1.1703953>.
- (1987). “The Statistics of Dimers on a Lattice”. In: *Classic Papers in Combinatorics*. Ed. by I. Gessel and G.-C. Rota. Boston, MA: Birkhäuser Boston, pp. 281–298. DOI: 10.1007/978-0-8176-4842-8_20. URL: https://doi.org/10.1007/978-0-8176-4842-8_20.
- Lackner, M., P. Regner, and B. Krenn (2023). “abcvoting: A Python package for approval-based multi-winner voting rules”. In: *Journal of Open Source Software* 8.81, p. 4880. DOI: 10.21105/joss.04880.
- Lackner, M. and P. Skowron (2023). *Multi-Winner Voting with Approval Preferences*. Springer International Publishing. DOI: 10.1007/978-3-031-09016-5. URL: <http://dx.doi.org/10.1007/978-3-031-09016-5>.
- Rey, S., F. Schmidt, and J. Maly (2025). *The (Computational) Social Choice Take on Indivisible Participatory Budgeting*. arXiv: 2303.00621 [cs.GT]. URL: <https://arxiv.org/abs/2303.00621>.
- Stolicki, D., S. Szufa, and N. Talmon (2020). *Pabulib: A Participatory Budgeting Library*. arXiv: 2012.06539 [cs.DC]. URL: <https://arxiv.org/abs/2012.06539>.
- Temperley, H. N. V. and M. E. Fisher (1961). “Dimer problem in statistical mechanics-an exact result”. In: *The Philosophical Magazine: A Journal of Theoretical Experimental and Applied Physics* 6.68, pp. 1061–1063. DOI: 10.1080/14786436108243366. eprint: <https://doi.org/10.1080/14786436108243366>. URL: <https://doi.org/10.1080/14786436108243366>.